Modul 3

Data Manipulation Language (DML)

A. Memasukkan Data (*Insert*)

Perintah *insert* berfungsi untuk mengisikan data ke dalam suatu tabel. Pengisian data ini bisa berlaku untuk satu record penuh atau untuk beberapa field saja di dalam record tersebut.

Sintak SQL-nya adalah sebagai berikut:

INSERT INTO nama_tabel (field ke-1, … , field ke-n)
VALUES (nilai_field ke-1, … ,nilai field ke-n);
Atau
INSERT INTO nama_tabel **VALUES** (nilai_field ke-1, … ,nilai field ke-n);

SOAL:

1. Isikan record ke dalam tabel DOKTER seperti tampilan berikut:

ID_DOKTER	NAMA_DOKTER	ALAMAT_DOKTER	TELEPON
D-001	dr. Suwanto Raharjo	Jl. Perintis 41 Purwokerto	081245745147
D-002	dr. Emi Susanti	Jl. Patriot 12 Purbalingga	
D-003	dr. Primania Rahma		0281-254785
D-004	dr. Gusti Prihananto	Jl. Kenanga 10 Sokaraja	

2. Isikan record ke dalam tabel PASIEN seperti tampilan berikut:

ID_PASIEN	NAMA_PASIEN	ALAMAT_PASIEN	KOTA_PASIEN
P-001	Suratno	Ds. Lebak Rt. 02/04	Banyumas
P-002	Prianti	Ds. Karangnanas Rt 01/03	Purwokerto
P-003	Susilo		Purwokerto
P-004	Wardoyo		Banyumas
P-005	Sri Rejeki	Ds. Purwosari Rt 03/01	Purwokerto

3. Isikan record ke dalam tabel OBAT seperti tampilan berikut:

KODE_OBAT	NAMA_OBAT	SATUAN-OBAT	HARGA_OBAT	STOK_OBAT
O-001	Paracetamol	Tablet	25.000	70
O-002	Cefadroxil	Kapsul	20.000	80
O-003	Somerol	Kapsul	15.000	50
O-004	Hiposetamol	Tablet	16.500	75
O-005	Amoxilin	Tablet	18.000	83
O-006	Lacoldin	Botol	27.000	68

4. Isikan record ke dalam tabel RESEP seperti tampilan berikut:

NO_RESEP	TGL_RESEP	ID_DOKTER	ID_PASIEN	TOTAL_RESEP
RSP-140401	01-04-2014	D-002	P-001	83.000
RSP-140402	01-04-2014	D-004	P-005	42.000
RSP-140403	12-04-2014	D-001	P-002	97.500

5. Isikan record ke dalam tabel DETAIL_RESEP seperti tampilan berikut:

NO_RESEP	KODE_OBAT	HARGA_OBAT	JUMLAH	SUBTOTAL
RSP-140401	O-001	25.000	1	25.000
RSP-140401	O-002	20.000	2	40.000
RSP-140401	O-005	18.000	1	18.000
RSP-140402	O-003	15.000	1	15.000
RSP-140402	O-006	27.000	1	27.000
RSP-140403	O-003	17.000	2	34.000
RSP-140403	O-004	16.500	1	16.500
RSP-140403	O-005	20.000	1	20.000
RSP-140403	O-006	27.000	1	27.000

B. Memperbaharui Data (*Update*)

Perintah *update* berfungsi untuk memperbaharui data lama dengan data yang baru. Jadi perintah update ini digunakan apabila sudah ada suatu data dalam tabel.

Sintak SQL-nya adalah sebagai berikut:

```
UPDATE    nama_tabel
SET       nama_field = data_baru
WHERE     nama_field = data_lama;
```

C. Menghapus Data (*Delete*)

Perintah *delete* berfungsi untuk menghapus data atau record. Penghapusan data dapat dilakukan secara keseluruhan maupun sebagian data saja. Pada penghapusan data sebagian gunakan klausa **Where** sebagai kondisi untuk memilih record mana yang akan dihapus. Perintah ini tidak akan berhasil jika data yang dihapus merupakan data reference tabel lain.

Sintak SQL-nya adalah sebagai berikut:

```
DELETE FROM  nama_tabel;
atau
DELETE FROM nama_tabel  WHERE kondisi;
```

D. Memilih Data/Seleksi Data (*Select*)

Perintah *select* berfungsi untuk menampilkan/menyeleksi data yang diinginkan dari sekumpulan data yang ada.

Sintak SQL untuk menampilkan seluruh field adalah sebagai berikut:

```
SELECT * FROM nama_tabel;
atau
SELECT * FROM nama_tabel  WHERE kondisi;
```

Sintak SQL untuk menampilkan field tertentu adalah sebagai berikut:

```
SELECT nama_field
FROM nama_tabel;
atau
SELECT nam_field
FROM nama_tabel  WHERE kondisi;
```

E. DISTINCT

Apabila kita ingin menampilkan data tetapi data yang sama tidak ditampilkan berulang-ulang maka kita bisa menggunakan kata kunci **DISTINCT**. Perintah ini dilakukan jika terdapat banyak data yang sama dalam satu kolom.

Sintaks umum:

```
SELECT DISTINCT (nama_field) FROM nama_tabel;
```

F. BETWEEN

Perintah ini jika kita ingin mengambil beberapa data yang terletak antara nilai dalam klausa **BETWEEN**. Hanya berlaku pada data yang bertipe numeric dan string.

Sintaks umum:

```
SELECT (nama_field) FROM nama_tabel WHERE (Kondisi)
BETWEEN [Batas Nilai 1] AND [Batas Nilai 2];
```

G. LIKE/NOT LIKE

LIKE digunakan untuk mengambil data dengan pencarian data yang memiliki bagian yang sama dalam klausa LIKE tersebut. Dengan menggunakan kondisi LIKE karakter apapun dapat dicari, dapat berupa satu, dua, atau semua karakter yang ada didalam string yang akan dicari. Ada beberapa hal yang harus diketahui dalam klausa LIKE, diantaranya:

Tanda % (*percent*)

Tanda % digunakan untuk mewakili semua karakter. Tanda ini bisa diletakkan di bagian depan, tengah maupun belakang.

Menampilkan data barang yang namanya memiliki diawali 'TRON'

```
SELECT * FROM barang WHERE nama LIKE 'TRON%';
```

Menampilkan data barang yang tidak berakhiran huruf 'O'

```
SELECT * FROM dokter WHERE nama NOT LIKE '%O';
```

Tanda _ (*underscore*)

Tanda _ digunakan untuk mewakili sebuah karakter. Tanda ini juga bisa diletakkan di bagian depan, tengah maupun belakang.

Misalnya dari tabel dokter yang ada, apabila kita ingin menampilkan data barang yang memiliki nama dengan karakter kedua adalah 'U', maka bisa digunakan perintah:

```
SELECT * FROM dokter WHERE nama LIKE '_U%';
```

H. Fungsi Agregate

Fungsi Agregate ialah fungsi – fungsi statistik yang ditetapkan pada baris – baris yang diseleksi. Fungsi Agregate sering digunakan untuk operasi perhitungan. Fungsi yang sering digunakan adalah:

AVG

Digunakan untuk mencari nilai rata-rata dari suatu field dalam sebuah tabel.

MAX

Digunakan untuk mencari nilai terbesar (maksimum) dari suatu field dalam sebuah tabel.

MIN

Digunakan untuk mencari nilai terkecil (minimum) dari suatu field dalam sebuah tabel.

SUM

Digunakan untuk mencari hasil jumlah dari suatu field dalam sebuah tabel.

COUNT

Digunakan untuk menghitung jumlah record atau baris data dari suatu field dalam sebuah tabel.

Sintaks SQL yang biasa dipakai:

```
SELECT AVG (nama_field) FROM nama_tabel;
SELECT MAX (nama_ field) FROM nama_tabel;
SELECT MIN (nama_ field) FROM nama_tabel;
SELECT SUM (nama_ field) FROM nama_tabel;
SELECT COUNT (nama_ field) FROM nama_tabel;
```

Contoh:

1. Menghitung jumlah barang

```
SELECT COUNT (*) FROM barang ;
SELECT COUNT (kd_ barang) FROM barang;
```

2. Menghitung jumlah stok barang terbesar

```
SELECT MAX (stok) FROM barang as maksimum_stok;
```

3. Menghitung jumlah stok barang terkecil

```
SELECT MIN (stok) FROM barang as minimum_stok;
```

4. Menghitung rata-rata pembelian barang

```
SELECT AVG (jumlah) FROM pembelian as rata_rata_beli;
```

5. Menghitung jumlah pembelian barang

```
SELECT SUM (jumlah) FROM pembelian as total_beli;
```

Selain contoh dasar di atas, fungsi agregate juga bisa dikombinasikan dengan penggunaan kondisi **WHERE**. Seperti contoh di bawah ini:

1. Menghitung jumlah barang yang telah terjual lebih dari 20

```
SELECT COUNT (kd_ barang) FROM pembelian WHERE jumlah>20;
```

2. Menghitung jumlah pembeli yang berasal dari Yogyakarta.

```
SELECT COUNT (id_pelanggan) FROM pelanggan WHERE
alamat='Yogyakarta';
```

I. Kolom Alias

Alias digunakan untuk memberi nama lain pada kolom – kolom hasil dari perintah SELECT. Alias ditulis setelah nama kolom atau ekspresi yang akan diganti / diberi nama dengan didahului keyword **AS**.

Sintaks dasar :

```
SELECT kolom1 AS alias1, kolom2 AS alias2, … , kolomN As aliasN FROM tabel;
```

Keterangan:

Jika kolom alias ada spasi maka gunakan tanda [ ] untuk kolom alias tersebut.

Contoh:

```
SELECT COUNT (kd_pasien) AS [Jumlah Pasien Dari Yogyakarta]
FROM pasien WHERE alamat='Yogyakarta';
```

J. Grouping

Grouping ialah proses pengelompokkan data yang dilakukan berdasarkan daftar kolom yang disebutkan dalam SELECT.

Sintaks dasar:

```
SELECT <field1,...,fieldn>,<Fungsi AGREGAT> (field) AS <nama alias>
FROM <nama tabel>
WHERE <kriteria (jika ada)>
GROUP BY <field1,...,fieldn>;
```

Catatan :

1. Semua kolom yang berada dalam daftar SELECT yang tidak berada dalam *group function* harus ditulis dalam klausa GROUP BY.
2. Semua pengelompokan mengabaikan nilai NULL di dalam kolom.

Contoh:

Menampilkan jumlah pelanggan dari tiap kota

```
SELECT alamat AS [Kota Pelanggan], COUNT(alamat) AS Jumlah
FROM pelanggan GROUP BY alamat;
```

K. Having

Memiliki fungsi yang sama dengan WHERE yaitu untuk menampung kriteria filtering. Perbedaannya adalah klausa HAVING akan digunakan apabila kondisi diberlakukan pada sebuah group atau fungsi agregat. HAVING digunakan untuk membatasi hasil pengolompokan (group).

Perbedaan antara WHERE dan HAVING :

- ✓ HAVING persyaratan pre-select atau sebelum data dipilih
- ✓ WHERE persyaratan post-select atau setelah data dipilih

Sintaks dasar :

```
SELECT <field1,...,fieldn>,<Fungsi AGREGAT> (field) AS <nama alias>
FROM <nama tabel>
WHERE <kriteria (jika ada)>
GROUP BY <field1,...,fieldn>
HAVING <kriteria>;
```

Contoh:

Menampilkan jumlah pelanggan dari tiap kota dimana jumlah pembelian > 50

```
SELECT alamat AS Kota_pelanggan, COUNT(alamat) AS Jumlah_beli FROM
pasien GROUP BY alamat
HAVING SUM (jumlah) > 50;
```

Soal praktikum:

1. Ubah stok obat Cefadroxil menjadi 65
2. Ubah harga obat Somerol menjadi 17.000 dan obat Amoxilin menjadi 20.000
3. Hapus data pasien bernama suratno
4. Hapus data dokter dengan nama dr. Primania Rahma
5. Tampilkan data dokter yang namanya tidak memiliki huruf 'S'
6. Tampilkan data pasien yang namanya berawalan 'S'
7. Tampilkan data obat dengan harga antara 10.000 sampai dengan 20.000
8. Tampilkan stok obat terkecil dari tabel obat
9. Tampilkan subtotal terbesar dari tabel detail resep
10. Tampilkan rata-rata subtotal dari tabel detail resep
11. Tampilkan jumlah total resep dari tabel resep
12. Tampilkan jumlah total obat yang dikeluarkan
13. Tampilkan jumlah data pada tabel detail resep
14. Tampilkan jumlah pasien yang beralamat di Purwokerto
15. Tampilkan data obat diurutkan berdasarkan nama obat